

UNIVERSIDADE FEDERAL DA BAHIA

INSTITUTO DE MATEMÁTICA
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

ANA CAROLINA SOKOLONSKI ANTON

AVALIANDO O USO DE SERVIDORES PARA TAREFAS APERIÓDICAS PARA FINS DE TOLERÂNCIA A FALHAS EM SISTEMAS COM ESCALONAMENTO DINÂMICO

Salvador

2005

ANA CAROLINA SOKOLONSKI ANTON

AVALIANDO O USO DE SERVIDORES PARA TAREFAS APERIÓDICAS PARA FINS DE TOLERÂNCIA A FALHAS EM SISTEMAS COM ESCALONAMENTO DINÂMICO

Monografia apresentada ao Curso de graduação em Ciência da Computação, Departamento de Ciência da Computação, Instituto de Matemática, Universidade Federal da Bahia, como requisito parcial para obtenção do grau de Bacharel em Ciência da Computação.

Orientador: *George Marconi de Araújo Lima*

Salvador

2005

RESUMO

Um sistema computacional é conhecido como sendo de tempo real se ele é projetado para reagir, dentro de prazos pré-estabelecidos, a estímulos provocados pelo ambiente. Vários tipos de sistemas de tempo real são considerados críticos, pois, caso algum destes prazos seja violado, graves acidentes podem ocorrer. Para garantir a correção temporal de tais sistemas, elaborados mecanismos de escalonamento devem estar presentes. Estes são responsáveis por gerar uma escala de execução das tarefas de tal forma que nenhum prazo de execução seja violado. Geralmente, o projeto de um mecanismo de escalonamento não considera os efeitos de possíveis falhas que podem ocorrer durante a execução de uma tarefa. Elaborar estes mecanismos para lidar com falhas não é simples e é tema corrente de pesquisas. A principal dificuldade está relacionada com o natural desconhecimento de quando e como tais falhas irão ocorrer. Investigar os efeitos que possíveis falhas têm no desempenho de mecanismos de escalonamento em Sistemas de Tempo Real Críticos é passo necessário para se projetar escalonadores que considerem tais efeitos. Para tanto, consideramos várias técnicas de escalonamento dinâmico, pois estas têm maior potencial para se adaptar em condições adversas. Em particular, investigamos mecanismos de escalonamento baseados em servidores de tarefas aperiódicas e determinamos o comportamento das mesmas considerando possíveis falhas durante a execução de tarefas. Um servidor de tarefa aperiódica é um mecanismo que visa executar as tarefas aperiódicas (não-críticas) controlando a interferência sobre as tarefas periódicas (críticas). Em princípio, este controle de interferência é uma característica desejável quando falhas são consideradas. Neste trabalho, por simulação, avaliamos vários servidores. Os resultados mostram que o uso de servidores não é adequado para executar tarefas críticas, devido a baixa capacidade de recuperação. Este estudo deve servir de base para que novas propostas sejam investigadas.

Palavras-chave: escalonamento dinâmico, sistema de tempo real, tolerância a falhas, servidor de tarefas aperiódicas.

LISTA DE FIGURAS

1 Escalonamento das tarefas j_1, j_2 e j_3 segundo o EDF 11

2 Escalonamento das tarefas j_1, j_2 e j_3 segundo o EDF, com DDS para executar as tarefas aperiódicas 15

3 Escalonamento das tarefas j_1, j_2 e j_3 segundo o EDF, com CUS para executar as tarefas aperiódicas 17

4 Escalonamento das tarefas j_1, j_2 e j_3 segundo o EDF, com TBS para executar as tarefas aperiódicas 19

5 Escalonamento das tarefas j_1, j_2 e j_3 segundo o EDF, com CBS para executar as tarefas aperiódicas 22

6 Modelo de escalonamento assumido 24

LISTA DE TABELAS

1 Tabela comparativa de desempenho de diversos servidores, feita a partir de simulação 26

SUMÁRIO

1 Introdução **7**

1.1 Sistemas de Tempo Real Críticos 7

1.2 Sistemas de Tempo Real Não-Críticos 8

1.3 Objetivo 9

2 Escalonamento de Tarefas Aperiódicas **10**

2.1 Modelo do Sistema 10

2.2 Técnicas para execução de tarefas aperiódicas 12

2.3 Servidores 13

2.3.1 Servidor *Deferrable* 14

2.3.2 Servidor de Constante Utilização 15

2.3.3 Servidor com Banda Total 18

2.3.4 Servidor com Banda Constante 20

3 Uso de servidores para Tolerância a Falhas **23**

3.1 Tolerância a Falhas 23

3.2 Modelo do Sistema Usado na Simulação 24

3.3 Descrição da Simulação 25

3.4 Avaliação dos Resultados 26

3.5 Considerações 27

4 Metodologia e Dificuldades Encontradas **28**

5 Considerações Finais **29**

Referências **30**

1 INTRODUÇÃO

Um Sistema de Tempo Real é um sistema computacional projetado para reagir a estímulos provocados pelo ambiente. Tais estímulos estão associados a uma ou mais tarefas. Tarefas são unidades de processamento, ou bloco de ações, seqüenciais que usam de forma concorrente um ou mais recursos computacionais do sistema. Cada tarefa possui um prazo máximo de execução chamado *deadline*. Num Sistema de Tempo Real, as tarefas devem apresentar correção lógica e temporal, ou seja, elas devem produzir resultados corretos, e devem cumprir seus *deadlines*.

Os Sistemas de Tempo Real podem ser caracterizados pelos seus *deadlines*. Esses *deadlines* devem ser obedecidos para garantir um comportamento correto. Quando a violação de *deadlines* não causa graves danos, chamamos o sistema de Sistema de Tempo Real Não-Crítico. Quando há a necessidade do cumprimento do *deadline* de alguma tarefa para que não ocorram catástrofes, por exemplo, esta tarefa é chamada de crítica. Um sistema com ao menos uma tarefa crítica é chamado de Sistema de Tempo Real Crítico (BURNS; WELLINGS, 2001).

Um importante mecanismo, responsável por garantir a correção temporal num Sistema de Tempo Real, seja ele crítico ou não, é o escalonador. De maneira simples, o escalonador deve escolher, baseado em alguma política de escalonamento, qual tarefa deve ser executada em função do tempo. Geralmente, o escalonador é projetado sem considerar os efeitos de falhas durante a execução de tarefas. Num Sistema de Tempo Real Crítico, isto muitas vezes é inaceitável. Neste trabalho, estamos interessados em investigar os efeitos que possíveis falhas têm em Sistemas de Tempo Real Críticos. A seguir, caracterizaremos melhor alguns conceitos, apresentaremos o objetivo desta monografia e descreveremos sua estrutura.

1.1 SISTEMAS DE TEMPO REAL CRÍTICOS

Sistemas de Tempo Real Críticos são geralmente caracterizados por um conjunto de tarefas bem comportado. Por exemplo, tarefas críticas têm um *período* de reativação conhecido (são periódicas) e cada uma possui um tempo máximo de execução determinado. Sabendo-se estas

características para todas as tarefas do sistema, pode-se, através da análise de escalonamento, prever, antes do sistema entrar em execução, se alguma tarefa poderá perder seu *deadline*. Obviamente, para tanto, é necessário que se conheça também o critério usado pelo escalonador para escolher qual tarefa irá executar.

Os critérios de escalonamento comumente usados são baseados em prioridades. O escalonador se limita a escolher a tarefa com mais alta prioridade dentre aquelas que estão prontas para executar. Prioridade representam, assim, a urgência de execução. Há duas classes de escalonamento baseados em prioridades, escalonamento estático e dinâmico. O escalonamento é dito estático se as prioridades das tarefas são fixadas em tempo de projeto. Já no escalonamento dinâmico, as prioridades das tarefas variam em função do estado do sistema durante sua execução.

Um das políticas de escalonamento dinâmico mais usadas é o EDF (*Earliest Deadline First*). A razão para esta popularidade é que, o EDF é simples de se implementar e ótimo. Ser ótimo significa que, se há um escalonamento possível, onde nenhuma tarefa perde o *deadline*, então o EDF também gera uma escala de execução onde nenhum *deadline* é violado (LIU, 2000).

Para entender melhor o escalonador EDF, considere duas tarefas, j_1 e j_2. Suponha que os *deadlines* de j_i ($i = 1,2$) seja D_i unidades de tempo após os respectivos instantes de liberação t_i. Assuma que $D_1 = 3$ e $D_2 = 5$ e que ambas as tarefas estão prontas para executar num dado instante t. O escalonador EDF escolherá a tarefa j_i que possui em t o menor *deadline* absoluto $d_i = t_i + D_i$. Por exemplo, considerando que $t_1 = 4$ e $t_2 = 1$, a tarefa escolhida para executar será j_2.

1.2 SISTEMAS DE TEMPO REAL NÃO-CRÍTICOS

Diferentemente dos Sistemas de Tempo Real Críticos, nos Sistemas de Tempo Real Não-Críticos, o conjunto de tarefas não é, em geral, bem comportado. Tarefas podem ser enfileiradas para execução sem que se saiba previamente qualquer informação sobre as mesmas. Geralmente, tarefas não-críticas são aperiódicas (não existe um período conhecido de ativação) e o seu tempo é desconhecido. Esse aspecto impede que se possa determinar se o sistema cumprirá todos os *deadlines*, por exemplo. De fato, nestas condições, pode inclusive haver uma sobrecarga no sistema, que geraria perda de *deadlines*. Um exemplo de um Sistema de Tempo Real Não-Crítico é um sistema multimídia. Note que, o tempo de execução de uma tarefa deste sistema depende do tipo e do conteúdo da imagem a ser exibida. No entanto, ao ocorrer uma perda

de *deadline* de uma tarefa, apenas haverá perda de qualidade da imagem que está sendo exibida.

Muitas vezes tarefas críticas e não-críticas co-existem num mesmo Sistema de Tempo Real. Neste caso, o mecanismo de execução deve ser capaz de "isolar" os efeitos das tarefas não-críticas sobre as críticas. Uma das abordagens comumente usadas são os servidores de tarefas aperiódicas. Estes consistem em um mecanismo que visa executar as tarefas aperiódicas controlando o impacto sobre a execução das tarefas periódicas.

1.3 OBJETIVO

Durante a execução de um sistema pode haver erros e estes podem levar o sistema a um estado inconsistente. Para que tais erros não levem o sistema a falhar, há várias técnicas de tolerância a falhas que se baseiam na detecção e recuperação de erros (LAPRIE, 1993). Tais técnicas são usadas em sistemas computacionais em geral, mas assumem um papel de extrema importância em Sistemas de Tempo Real Críticos.

Suponha por exemplo que haja um erro durante a execução de uma tarefa crítica. Após o erro ser detectado, o sistema deve executar uma rotina de recuperação para mantê-lo funcionando corretamente. Esta rotina, pode ser modelada, por exemplo, por uma tarefa *esporádica*, que é acionada, eventualmente, em função da detecção de erros. Porém, tal tarefa de recuperação, no caso dos Sistemas de Tempo Real Críticos, deve terminar com sucesso respeitando o *deadline* da tarefa original. Este detalhe dificulta a implementação de tolerância a falhas em dois aspectos. Primeiro, é necessário que o sistema seja reativo o bastante para que não haja perda de *deadlines* em cenários de erros. Segundo, ao recuperar uma tarefa, o escalonamento do sistema deve prover "isolamento"adequado para que a tarefa de recuperação não cause perda de *deadlines* de tarefas que não falharam.

Uma alternativa para prover tal isolamento é usar servidores, tal como os usados quando há tarefas críticas e não-críticas co-existindo. Baseados nesta observação, objetivamos, neste trabalho, investigar os efeitos do uso de técnicas de escalonamento baseadas em servidores para prover tolerância a falhas em Sistemas de Tempo Real Crítico.

2 ESCALONAMENTO DE TAREFAS APERIÓDICAS

Num Sistema de Tempo Real podem existir três tipos de tarefas: tarefas periódicas, tarefas aperiódicas e tarefas esporádicas. Tarefas *periódicas* são aquelas que são reativadas em intervalos regulares de tempo chamados *períodos*. Tarefas *aperiódicas* são tarefas que não têm período definido, assim podem ser reativadas a qualquer momento. Sua ativação é aleatória por estar, normalmente, vinculada a eventos internos ou externos do sistema que não possuem período fixo, nem qualquer indício de quando irão ocorrer. Tarefas *esporádicas* são tarefas que não têm período definido, assim como as aperiódicas, porém um intervalo mínimo entre suas ativações é conhecido. Normalmente, tarefas periódicas são críticas e aperiódicas são não-críticas.

Neste capítulo, definiremos um modelo de escalonamento que utilizaremos no restante da monografia. Descreveremos algumas técnicas para execução de tarefas aperiódicas em Sistemas de Tempo Real Críticos, porém nos concentraremos na técnica conhecida como servidor de tarefas aperiódicas, ou simplesmente, servidor. Apresentaremos alguns dos servidores existentes, suas vantagens e desvantagens, comparando-os.

2.1 MODELO DO SISTEMA

Consideramos um Sistema de Tempo Real uniprocessado com preempção, que pode ter em sua fila de execução dois tipos de tarefas, periódicas e aperiódicas.

Usaremos a seguinte notação ao longo deste relatório: letras minúsculas e maiúsculas são consideradas diferentes. Tarefas serão representadas por j_i, que é a tarefa de identificação i. Instâncias de tarefas serão representadas por $j_{i,k}$, que é a tarefa de identificação i, que está em sua *k-ésima* ativação. Seus atributos serão denotados da seguinte forma: p_i é o período da tarefa j_i. c_i é o tempo máximo de execução da tarefa j_i. D_i é o *deadline* relativo da tarefa j_i. $d_{i,k}$ é o *deadline* absoluto da tarefa j_i que está em seu período k. $u_i = \frac{c_i}{p_i}$ o percentual de utilização do processador solicitado pela tarefa j_i.

O modelo de escalonamento usa o algoritmo EDF para escalonar suas tarefas. Um exemplo pode ser visto na figura 1, onde j_1, j_2 e j_3 são tarefas periódicas, a linha horizontal é a linha temporal, j_1 tem $c_1 = 3$ e $D_1 = 10$, j_2 tem $c_2 = 4$ e $D_2 = 12$ e j_3 tem $c_3 = 2$ e $D_3 = 7$. As três tarefas são iniciadas no instante zero. Os quadrados cheios representam tarefas em execução. Os quadrados tracejados indicam que há uma espera na fila de prontos. Neste momento, a tarefa j_3 é a mais prioritária por apresentar menor *deadline* absoluto, $d_{3,1} = 7$. No instante 2 a tarefa j_3 termina de executar e a tarefa j_1 passa a ser a tarefa mais prioritária. No instante 5, quando a tarefa j_1 terminou sua execução, j_2 passa a executar. No instante 7 a tarefa j_3 é reiniciada, porém seu *deadline* agora é 14, assim ela espera a tarefa j_2, que naquele momento é a mais prioritária, com *deadline* 12, terminar de executar. Esse processo é perpetuado por serem tarefas periódicas.

Assumiremos que as tarefas não compartilham recursos (são independentes). Nestas condições, sabe-se que se $\sum u_i \leq 1$, o sistema é escalonável (LIU, 2000). Em outras palavras, nenhuma tarefa perde o *deadline*. Apresentaremos diversas abordagens desenvolvidas para escalonar tarefas aperiódicas e suas implicações.

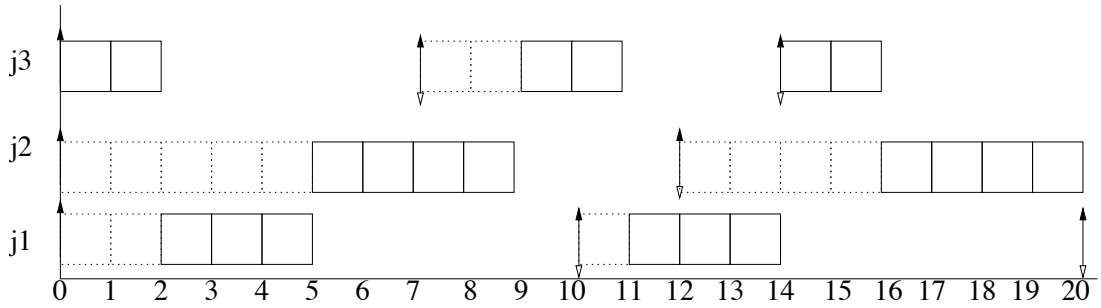


Figura 1: Escalonamento das tarefas j_1, j_2 e j_3 segundo o EDF

A ilustração apresentada na figura 1 contempla apenas tarefas periódicas. Se tarefas aperiódicas forem consideradas, usar apenas EDF pode não ser suficiente. Por exemplo, se uma tarefa não-crítica chega no instante 9 e possui *deadline* absoluto $d_{4,1} = 14$ e $c_4 = 5$, $j_{2,2}$ perderá seu *deadline*.

2.2 TÉCNICAS PARA EXECUÇÃO DE TAREFAS APERIÓDICAS

O escalonador deve tentar executar a tarefa aperiódica o mais rápido possível, sem causar perdas de *deadlines* de tarefas periódicas. Este aspecto deve ser considerado na escolha de uma técnica de escalonamento. Tarefas críticas não têm a urgência de execução, é necessário apenas

que se cumpra os seus *deadlines*.

Desta forma, quando há tarefas periódicas e aperiódicas no mesmo sistema, o que se busca é minimizar o tempo de resposta das segundas e ao mesmo tempo não violar os *deadlines* das primeiras. Existem várias técnicas de escalonamento que se prestam a atender ambos os objetivos (LIU, 2000).

Uma abordagem que pode ser usada é conhecida como execução em *background*. De acordo com esta abordagem, tarefas aperiódicas somente são escalonadas e executadas quando não existem tarefas periódicas e/ou esporádicas prontas para executar. Esse método é simples de implementar e sempre apresentará um escalonamento correto, não provocando perdas de *deadlines* de tarefas periódicas. É, porém, um método pessimista. As tarefas aperiódicas podem esperar muito até começarem a executar.

A técnica de execução por roubo de folga usa um algoritmo para calcular as folgas das tarefas escalonadas. Folga é o espaço em que o sistema encontra-se ocioso entre o momento que uma tarefa termina de executar e o seu *deadline*. Após avaliar a folga do sistema, se houver folga, obtemos uma quantidade de tempo total que todas as tarefas periódicas podem ser adiadas. Esse tempo é utilizado pela tarefa aperiódica no momento de sua ativação, causando um adiamento da execução das tarefas periódicas. Desta forma, a tarefa aperiódica é executada, não precisando esperar até que as tarefas periódicas adiadas terminem suas execuções. Caso não haja folga, quando a tarefa aperiódica chegar, ela deverá aguardar até que a folga seja detectada. Essa abordagem é extremamente custosa, devido a complexidade do algoritmo de cálculo de folgas (LIU, 2000). O roubo de folga, geralmente, se mostra inviável na prática.

Uma quarta abordagem é através do uso de servidores. A idéia é proporcionar um meio das tarefas aperiódicas executarem sem afetar as tarefas periódicas que estão no sistema e aproveitando o máximo do tempo disponível. Isto é feito através da inclusão de uma tarefa extra no sistema, chamada de servidor. O servidor é uma tarefa virtual e sua função é escalonar as tarefas aperiódicas. Por exemplo, quando o servidor é selecionado para executar pelo escalonador, o que se executa na verdade são tarefas aperiódicas que estão prontas para executar.

Existem vários tipos de servidores. As seções seguintes descrevem alguns dos mais usados para escalonamento baseado em EDF.

2.3 SERVIDORES

Um servidor de tarefa aperiódica é um mecanismo que visa executar as tarefas aperiódicas (não-críticas) controlando a interferência sobre as tarefas periódicas (críticas). O servidor é

incluído no teste de escalonabilidade do sistema como uma tarefa periódica qualquer. Por exemplo, pode-se definir o servidor como uma tarefa virtual de período p_s e tempo máximo de execução c_s. Sua prioridade dependerá da política de escalonamento adotada e/ou das prioridades das tarefas aperiódicas que estão em sua fila. Isso faz com que as tarefas aperiódicas não esperem demais para executar, ao mesmo tempo em que não afeta a escalonabilidade do sistema. Afinal, já fora avaliado, em tempo de projeto, a viabilidade da execução de uma tarefa de período p_s e tempo máximo de execução c_s.

Essa técnica, torna desnecessários cálculos demasiados para avaliação da escalonabilidade do sistema. Cada tarefa aperiódica que chega na fila de prontos do servidor não afeta o custo de processamento do sistema. Os servidores poderão ter prioridade fixa ou variável. Enfocaremos em servidores com prioridade variável.

Os servidores podem assumir alguns estados que serão listados a seguir:

Servidor backlogged: Quando existem tarefas aperiódicas para executar, ou seja, a fila associada ao servidor não está vazia, e o servidor tem *carga* para executar.

Servidor ocioso: Quando a fila de tarefas aperiódicas encontra-se vazia, ou quando a fila de tarefas aperiódicas não encontra-se vazia e o servidor não tem *carga* para executar.

Servidor ativo ou executando: Quando uma tarefa aperiódica está executando.

Pode-se dividir os diferentes tipos de servidores em duas classes: servidores *polling* e servidores com preservação de tamanho de banda. Servidor *polling* (p_s, c_s) é uma tarefa periódica, onde p_s é o período e c_s é o tempo máximo de execução (*carga*). Quando executado, o servidor examina a sua fila de prontos, que consiste numa fila de tarefas aperiódicas. Se esta não estiver vazia, a primeira tarefa da fila é executada durante o tempo máximo, c_s. Se a fila estiver vazia, o servidor é colocado em estado ocioso e somente retorna ao escalonamento na próxima ativação, após aguardar o término do período p_s. Essa abordagem apresenta problemas de desempenho. No momento que uma tarefa aperiódica chega na fila logo após o servidor entrar em estado ocioso. Nesta situação, a tarefa aperiódica terá que esperar até a próxima ativação do servidor para executar, tornando sua espera mais longa que o necessário.

O servidor com preservação de tamanho de banda, como o nome indica, é um servidor que preserva sua carga para num tempo futuro executar tarefas aperiódicas. As principais diferenças entre os servidores com preservação de tamanho de banda estão relacionadas às regras de consumo e recarga do servidor. Um servidor com preservação de tamanho de banda está *backlogged* quando existe uma *carga* disponível diferente de zero e há tarefas em sua fila de prontos. Esse

servidor entra em estado ocioso quando a *carga* fora esgotada.

2.3.1 SERVIDOR *DEFERRABLE*

Um servidor *deferrable* (DDS - *Dynamic Deferrable Server*) é o tipo mais simples de servidor com preservação de tamanho de banda. Como o *polling*, o sua *carga* é realizada periodicamente a cada intervalo p_s. Diferente do *polling*, o servidor *deferrable* mantém a *carga* inalterada mesmo quando o servidor está ocioso. Ao detectar que a fila de tarefas aperiódicas está vazia, o servidor não executa e mantém sua *carga* intacta. Ao chegar uma tarefa aperiódica na fila, o servidor é acionado e executa a tarefa até a *carga* esgotar. Seu *deadline*, d_s, está sempre definido (LIU, 2000) e é atualizado a cada período p_s.

Regras de Consumo: O tempo de execução é consumido a uma taxa de uma execução por unidade de tempo. A *carga* só poderá ser consumida com o servidor *deferrable* em estado ativo.

Regras de Recarga: A *carga* é realizada para c_s no início de cada período.

Para melhor ilustrarmos o funcionamento do DDS, considere a figura 2, onde j_1, j_2 e j_3 são tarefas periódicas. Os atributos destas tarefas são $c_1 = 2$ e $D_1 = 10$, $c_2 = 4$ e $D_2 = 12$, $c_3 = 2$ e $D_3 = 7$. O servidor DDS tem $c_s = 2$ e $D_s = 10$. O sistema funciona idêntico ao EDF mostrado na figura 6 até o momento 8, quando chega uma tarefa aperiódica j_a com tempo máximo de execução, c_a, igual a 4. A partir deste momento o servidor DDS é escalonado junto com as tarefas periódicas do sistema, pois passa de estado ocioso para *backlogged*, com *deadline* absoluto $d_{s,1} = 10$ e com 2 unidades de tempo para executar, que é seu tempo máximo de execução. Quando sua prioridade é máxima, o mesmo executa durante 2 unidades de tempo. Apesar da tarefa aperiódica ainda não ter concluído sua execução, pois precisa executar 4 unidades de tempo, o DDS passa ao estado ocioso, e somente é escalonado no período seguinte com $d_{s,2} = 20$, executando mais 2 unidades de tempo. Com isto, conclui a execução da tarefa aperiódica j_a.

Escalonabilidade do Deferrable: No pior caso, podem existir duas execuções consecutivas do servidor *deferrable*. Basta, para isso, que ele comece a executar uma tarefa aperiódica a c_s unidades de tempo da sua próxima ativação, com a *carga* igual a c_s e que a fila de prontos do servidor não esteja vazia. Assim, o *deferrable* ocupará duas vezes c_s unidades de tempo do processador. Isto significa que o servidor pode atrasar a execução de tarefas menos prioritárias, no pior caso, $2c_s$ unidades de tempo. De uma maneira geral, um sistema com n tarefas periódicas, preemptivas e independentes, que contém um servidor *deferrable* e segue a política

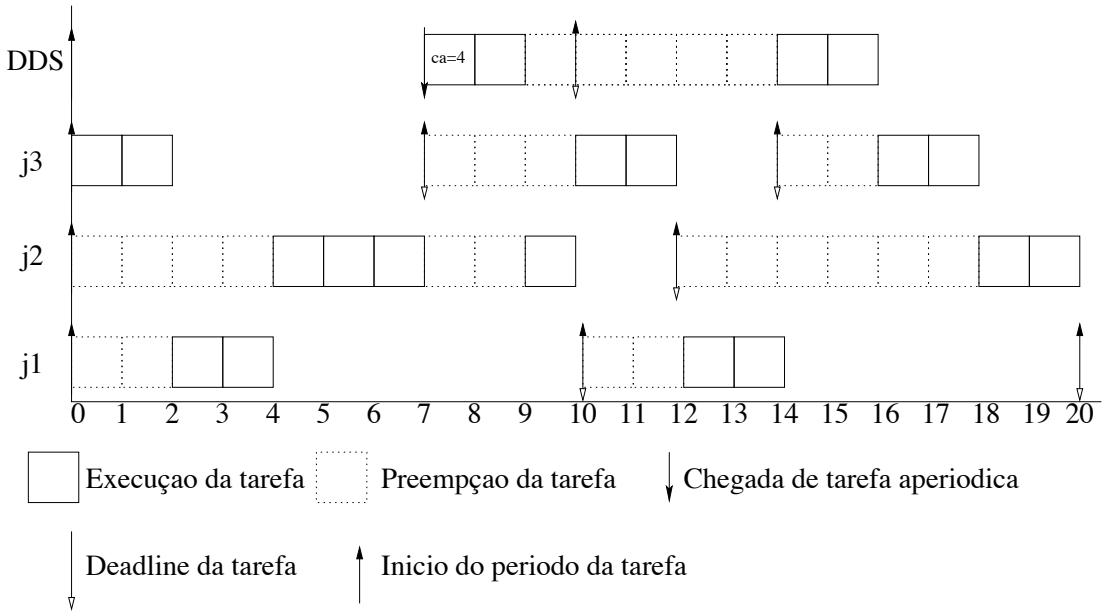


Figura 2: Escalonamento das tarefas j_1, j_2 e j_3 segundo o EDF, com DDS para executar as tarefas aperiódicas

de escalonamento EDF é escalonável se (LIU, 2000):

$$\sum_{k=1}^{n} \frac{c_k}{\min(d_k, p_k)} + u_s(1 + (\frac{p_s - c_s}{d_i})) \leq 1. \tag{2.1}$$

2.3.2 SERVIDOR DE CONSTANTE UTILIZAÇÃO

O servidor de constante utilização (CUS - *Constant Utilization Server*) (DENG; SUN, 1997) é definido pelo seu tamanho u_s, que representa a fração do processador reservada ao servidor. Assim, como o servidor *deferrable*, o *deadline*, d_s, do CUS está sempre definido.

O CUS se torna elegível para execução, quando sua *carga* é maior do que zero. O servidor se torna inelegível para execução quando a *carga* termina, voltando a ser elegível quando a *carga* é realizada novamente.

Regras de Consumo: A *carga* do servidor somente é consumida quando o servidor está em execução, decrementando uma por unidade de tempo, até que a *carga* termine. Para tanto, o servidor deve estar executando com permissão do escalonador EDF para executar, segundo sua prioridade. Alguma tarefa deve estar na fila de tarefas aperiódicas, para que o servidor calcule seu *deadline* a partir do *deadline* da tarefa, conseqüentemente, tenha uma prioridade definida.

Regras de Recarga: A *carga* do CUS é feita e seu *deadline* é modificado segundo as seguintes

regras: Seja c_s o tempo máximo de execução de um servidor de constante utilização e d_s o seu respectivo *deadline*. Seja j_a uma tarefa aperiódica, c_a seu tempo máximo de execução e D_a o seu *deadline* relativo. Inicialmente, $carga = 0$ e $d_s = \infty$. Quando j_a chega no tempo t estando a fila de tarefas aperiódicas vazia, temos que se $t \leq d_s$, o servidor não precisa fazer nada. Se $t \geq d_s$, o servidor atribui um novo valor a d_s e a *carga*:

$$d_s = t + \frac{c_a}{u_s} \tag{2.2}$$

e $carga = c_a$.

Ao atingir o *deadline*, $d_{s,1}$, o servidor terá seu *deadline* modificado segundo as seguintes regras: se o servidor está em estado *backlogged*, seja a tarefa aperiódica j_a, que se encontra no topo da fila de prontos do servidor, com tempo máximo de execução c_a e *deadline* relativo d_a. O *deadline* do servidor é modificado para:

$$d_s = d_s + \frac{c_a}{u_s} \tag{2.3}$$

e sua *carga* é modificada para c_k. Se o servidor estiver em estado ocioso, nada deverá ser feito (DENG; SUN, 1997).

Um servidor de constante utilização é dito *backlogged* quando sua *carga* não é nula. Quando o servidor está *backlogged*, ele é escalonado junto com as tarefas periódicas, segundo o algoritmo EDF.

Como pode ser notado, um servidor CUS sempre está fornecendo *carga* suficiente para completar a tarefa que está no topo da fila de prontos, a cada vez que a *carga* é realizada. O *deadline* é modificado de tal forma, que a utilização do processador pelo servidor se mantém igual a u_s.

O CUS emula uma tarefa esporádica, isto é, se comporta como uma tarefa esporádica com utilização instantânea constante, por isso possui esse nome.

Para um melhor entendimento do funcionamento do CUS, considere a figura 3, onde j_1, j_2 e j_3 são tarefas periódicas. Os atributos destas tarefas são $c_1 = 2$ e $D_1 = 10$, $c_2 = 4$ e $D_2 = 12$, $c_3 = 2$ e $D_3 = 7$. O servidor CUS tem $u_s = 0,2$. O escalonamento é idêntico ao mostrado na figura 6 até o instante 7, quando chega uma tarefa aperiódica j_a com tempo máximo de execução, c_a, igual a 4.

A partir deste momento, o servidor CUS é escalonado junto com as tarefas periódicas do sistema, pois passa de estado ocioso para *backlogged*. Quando a tarefa j_a chega, o *deadline* do servidor é calculado para 27, pois $d_{s,1} = 7 + (\frac{4}{0,2}) = 27$. No instante 11, quando a tarefa

j_b chega, o *deadline* do CUS não é alterado e j_b é colocada em sua fila de tarefas aperiódicas. Quando sua prioridade é máxima, o mesmo executa enquanto tiver mais alta prioridade e tiver *carga* a executar, ou seja, tarefas em sua fila de prontos.

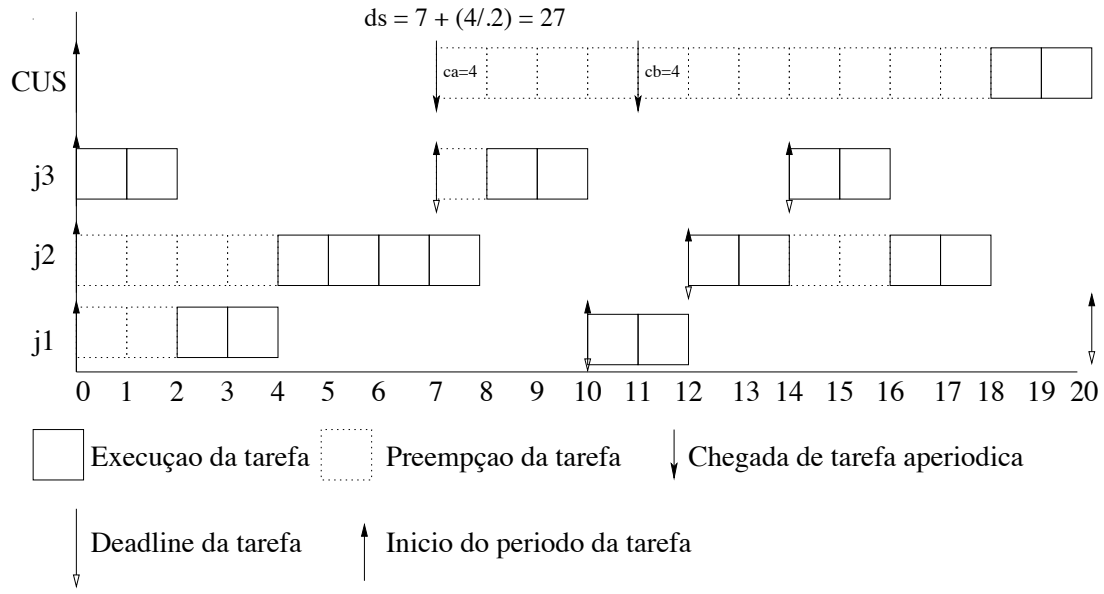


Figura 3: Escalonamento das tarefas j_1, j_2 e j_3 segundo o EDF, com CUS para executar as tarefas aperiódicas

Escalonabilidade do CUS: Num sistema com uma série de tarefas periódicas com taxa de utilização do processador u_p e um servidor CUS com taxa de utilização do processador u_s. Dizemos que o sistema é escalonável, segundo o algoritmo EDF, se e somente se, a equação

$$(u_s + u_p) \leq 1 \tag{2.4}$$

(SPURI; BUTTAZZO, 1996) for válida.

2.3.3 SERVIDOR COM BANDA TOTAL

O servidor com preservação total de tamanho de banda (TBS - *Total Bandwidth Server*) (SPURI; BUTTAZZO, 1996) melhora o aproveitamento do CUS ,através da utilização do tempo de *background* do processador não utilizado pelas tarefas periódicas. No momento em que a *carga* é esgotada, caso o servidor esteja *backlogged*, ele é imediatamente recarregado, caso contrário, a *carga* é realizada assim que o servidor atingir o estado *backlogged*. A determinação dos *deadlines* tem que ser feita para cada tarefa, visando que a total utilização do processador

nunca exceda o valor máximo da utilização do processador do servidor, u_s. Quando a *k-ésima* tarefa chegar, no tempo t, o servidor recebe a seguinte definição do seu *k-ésimo deadline*:

$$d_{s,k} = \max(t, d_{s,k-1}) + (\frac{c_a}{u_s}) \tag{2.5}$$

, onde c_a é o tempo máximo de execução da tarefa aperiódica que encontra-se no topo da fila, u_s é a utilização do processador reservada ao TBS. Por definição, $d_{s,0} = \infty$.

A tarefa é inserida na fila de prontos do servidor. O sistema, posteriormente, escalonada segundo EDF o servidor TBS, como qualquer outra tarefa periódica que esteja presente no sistema. Note que podemos manter parte da banda do servidor reservada para outras tarefas, simplesmente tomando o máximo entre t_k e d_{k-1}.

O algoritmo do servidor TBS é, essencialmente, o mesmo do servidor CUS. A única diferença entre eles é que no algoritmo TBS, quando uma tarefa que estava no topo da fila de prontos do servidor termina, tem sua *carga* imediatamente recarregada, se a fila de prontos não estiver vazia. Enquanto o CUS, na mesma situação, não modifica o valor da *carga*. Desta forma, pode-se usar o tempo de *background*, quando não existe tarefas periódicas para executar, para diminuir o tempo de resposta das tarefas aperiódicas.

Intuitivamente, a definição dos *deadlines* é tal que, em cada intervalo de tempo, a razão alocada pelo EDF para as tarefas aperiódicas nunca exceda a utilização do processador do TBS, u_s. Dessa forma, a utilização do processador das tarefas aperiódicas é sempre, no máximo, u_s.

Regras de Consumo: Com regras de consumo idênticas às do servidor CUS.

Regras de Recarga: Inicialmente, $carga = 0$ e $d_{s,0} = \infty$. Quando uma tarefa aperiódica com tempo de execução c_a chega no tempo t, com a fila de tarefas aperiódicas do servidor TBS vazia, igualamos $d_{s,k}$ a:

$$d_{s,k} = \max(d_{s,k-1}, t) + \frac{c_a}{u_s} \tag{2.6}$$

e $carga = c_a$.

Quando o servidor completa sua tarefa aperiódica corrente, esta é removida da fila de prontos. Se o servidor está em estado *backlogged*, seja a tarefa j_a a nova tarefa aperiódica que se encontra no topo da fila de prontos do servidor com tempo máximo de execução c_a, o *deadline* $d_{s,k}$ do servidor será igualado a $d_{s,k-1} + \frac{c_a}{u_s}$ e sua *carga* será igualada a c_a. Se o servidor estiver ocioso, nada será feito.

Para entendermos melhor o funcionamento do TBS, podemos observar a figura 4, onde

j_1, j_2 e j_3 são tarefas periódicas. Os atributos destas tarefas são $c_1 = 2$ e $D_1 = 10$, $c_2 = 4$ e $D_2 = 12$, $c_3 = 2$ e $D_3 = 7$. O servidor TBS tem $u_s = 0,2$. Quando chega uma tarefa aperiódica j_a, no instante 1, com tempo máximo de execução, c_a, igual a 2, o *deadline* do servidor TBS é calculado, $d_{s,1} = t + (\frac{c_a}{u_s}) = 1 + (\frac{2}{0,2}) = 11$. O servidor TBS passa de estado ocioso para *backlogged* e é escalonado junto com as tarefas periódicas do sistema. No instante 4 o servidor TBS é a tarefa de mais alta prioridade e este executa duas unidades de tempo, concluindo a execução da tarefa j_a. A tarefa j_b chega no instante 7, com $c_b = 4$. j_b é colocada em sua fila de tarefas aperiódicas. Como a fila encontra-se vazia, o *deadline* do servidor é recalculado, $d_{s,2} = \max(t, d_{s,1}) + (\frac{c_b}{u_s}) = \max(7,11) + (\frac{4}{0,2}) = 31$. O TBS passa a ter baixa prioridade e as tarefas periódicas executam. Quando sua prioridade for máxima, o mesmo executa enquanto tiver mais alta prioridade e tiver *carga* a executar, ou seja, tarefas em sua fila de prontos. Sempre recalculando o *deadline* do servidor para não deixar que o mesmo cause perda de *deadlines* de tarefas periódicas.

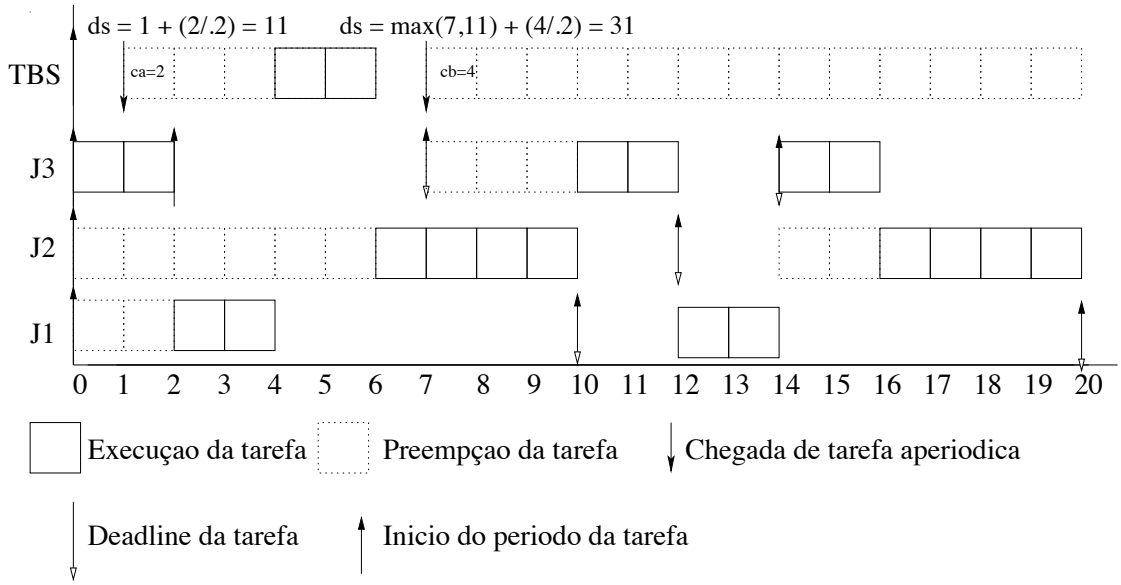


Figura 4: Escalonamento das tarefas j_1, j_2 e j_3 segundo o EDF, com TBS para executar as tarefas aperiódicas

Escalonabilidade do TBS: O TBS foi definido como uma maneira de garantir que a utilização do processador pelas tarefas aperiódicas nunca exceda u_s, tentando obter uma total utilização do processador. Em cada intervalo de tempo $[t_1, t_2]$, se T_a é o tempo total de execução demandado pelas tarefas aperiódicas que chegaram em t_1, ou depois de t_1, e têm *deadline* menor ou igual a t_2, então a equação

$$T_a \leq \frac{(t_2 - t_1)c_s}{p_s} \tag{2.7}$$

é válida (SPURI; BUTTAZZO, 1996). Assim, é possível mostrar que, dada uma série de n tarefas

periódicas com utilização do processador u_p e um TBS com utilização do processador u_s.

O sistema com escalonamento EDF usando o TBS para servir suas tarefas aperiódicas, é escalonável se, e somente se, a equação 2.4 for válida (SPURI; BUTTAZZO, 1996).

O tempo de uso de processador do TBS é próximo ao tempo de uso do CUS, pois se a fila não estiver vazia, o servidor continua a recarregar a *carga* e a executar as tarefas aperiódicas sem ter que esperar pelo próximo *deadline*. Logo, ele permite e execução de mais tarefas aperiódicas e provoca uma espera menor às tarefas aperiódicas para executá-las, diminuindo o número de *deadlines* perdidos.

2.3.4 SERVIDOR COM BANDA CONSTANTE

O servidor com preservação de tamanho de banda constante (CBS - *Constant Bandwidth Server*) (ABENI; BUTTAZZO, 2004) garante que se u_s é uma fração do tempo do processador designada para o servidor, esta contribuição para a total utilização não é maior do que u_s, mesmo na presença de sobrecarga no sistema. Esta propriedade não é válida para o TBS, nem para o CUS, nos quais contribuições são limitadas a u_s se, e somente se, assume que todas as tarefas servidas executam não mais que o declarado.

A idéia básica em torno do CBS é que quando uma nova tarefa entra no sistema, a ela é atribuído um escalonamento adequado ao seu *deadline* e esta é inserida na fila de prontos do EDF. Se a tarefa tentar executar mais tempo do que o permitido, seu *deadline* é adiado, diminuindo sua prioridade, eliminando a interferência desta tarefa nas demais. Notemos que, pelo adiamento do *deadline*, a tarefa permanecerá elegível a execução. Desta forma, o CBS se comporta como um algoritmo de conservação, explorando a folga disponível no sistema.

Se uma série de *n* tarefas é servida por apenas um servidor, todas as tarefas desta série compartilharão a mesma banda, não existindo isolamento entre elas. No entanto, todas as demais tarefas do sistema estarão isoladas das tarefas desta série.

Para não perder qualquer *deadline* de tarefas críticas, a determinação de regras para adaptação do *deadline* do servidor têm que ser cuidadosamente avaliada.

Um servidor CBS se caracteriza pelo par ordenado (c_s, p_s), onde c_s é o máximo valor de *carga* disponível ao servidor e p_s é o período de recarga do servidor. A razão $u_s = \frac{c_s}{p_s}$ é denominada como a utilização total do processador do servidor. A cada ativação k do servidor, um *deadline*, $d_{s,k}$, é definido. Inicialmente, $d_{s,0} = \infty$.

Regras de Consumo: Para cada tarefa atendida, $j_{i,l}$, é determinado um *deadline* dinâmico,

$d_{i,l}$, igual ao *deadline* corrente do servidor. Sempre que uma tarefa da fila de prontos do servidor executa, a *carga* é decrementada na mesma quantidade de unidades de tempo.

Regras de Recarga: Quando $carga = 0$, a *carga* do servidor é recarregada para o máximo valor, c_s, e um novo *deadline* é determinado para o servidor através da equação

$$d_{s,k+1} = d_{s,k} + p_s. \tag{2.8}$$

Podemos perceber que não existem intervalos finitos de tempo nos quais a *carga* seja nula, pois sempre que a *carga* atinge o valor nulo é atribuído um novo valor maior que zero, que é o valor máximo de *carga* do servidor, c_s.

Um CBS está em estado *backlogged* no tempo t se existirem tarefas em sua fila de prontos, isto é, se existir tarefas $j_{i,l}$ tal que $t_{i,l} \leq t \leq d_{i,l}$, onde $t_{i,l}$ é o tempo de chegada da tarefa $j_{i,l}$. Um CBS é dito ocioso caso não esteja em estado *backlogged* ou ativo.

Quando uma tarefa $j_{i,l}$ chega e o servidor está ativo, ela é colocada na fila de prontos do servidor de acordo com a política de escalonamento da mesma. Quando uma tarefa $j_{i,l}$ chega e o servidor está ocioso, se a *carga* for $\geq (d_{s,k} - t)u_s$, então o servidor gerará um novo *deadline* $d_{s,k+1} = t + p_s$ e a *carga* será realizada, caso contrário, a tarefa será atendida com o último *deadline* do servidor $d_{s,k}$, usando a *carga* corrente.

Quando uma tarefa termina de executar, a próxima tarefa que estiver no topo da fila de prontos, caso exista, é atendida usando a *carga* e o *deadline* corrente. Se não existirem tarefas na fila, o servidor passa ao estado ocioso.

Um exemplo de CBS pode ser visto na figura 5, onde j_1, j_2 e j_3 são tarefas periódicas. Os atributos destas tarefas são $c_1 = 2$ e $D_1 = 10$, $c_2 = 4$ e $D_2 = 12$, $c_3 = 2$ e $D_3 = 7$. O servidor CBS tem $c_s = 2$ e $p_s = 10$, ou seja, é formado pelo par ordenado $(2, 10)$. O escalonamento é idêntico ao mostrado na figura 6 até o instante 7, quando uma tarefa aperiódica j_a com tempo máximo de execução, c_a, igual a 4 chega ao sistema. Neste instante o CBS é escalonado junto com as tarefas periódicas do sistema, pois passa de estado ocioso para *backlogged*.

Quando a tarefa j_a chega, a *carga* do servidor é comparada com $(d_{s,1} - t)u_s = (10 - 7)0,2 = 0,8$, como a *carga* é maior que o inteiro mais próximo de $0,8$, que é 1, um novo *deadline* deve ser calculado. $d_{s,2} = t + p_s = 7 + 10 = 17$. No instante 10, o CBS passa a ser a tarefa mais prioritária e começa a executar j_a, porém no instante 12 sua *carga* atinge zero e seu *deadline* é recalculado para $d_{s,3} = d_{s,2} + p_s = 17 + 10 = 27$. Logo, o CBS deixa de ser o mais prioritário e é preemptado para a execução das tarefas periódicas. Este só voltará a executar as tarefas aperiódicas de sua fila, quando tornar a ser a tarefa mais prioritária do sistema, obedecendo

suas regras supra citadas.

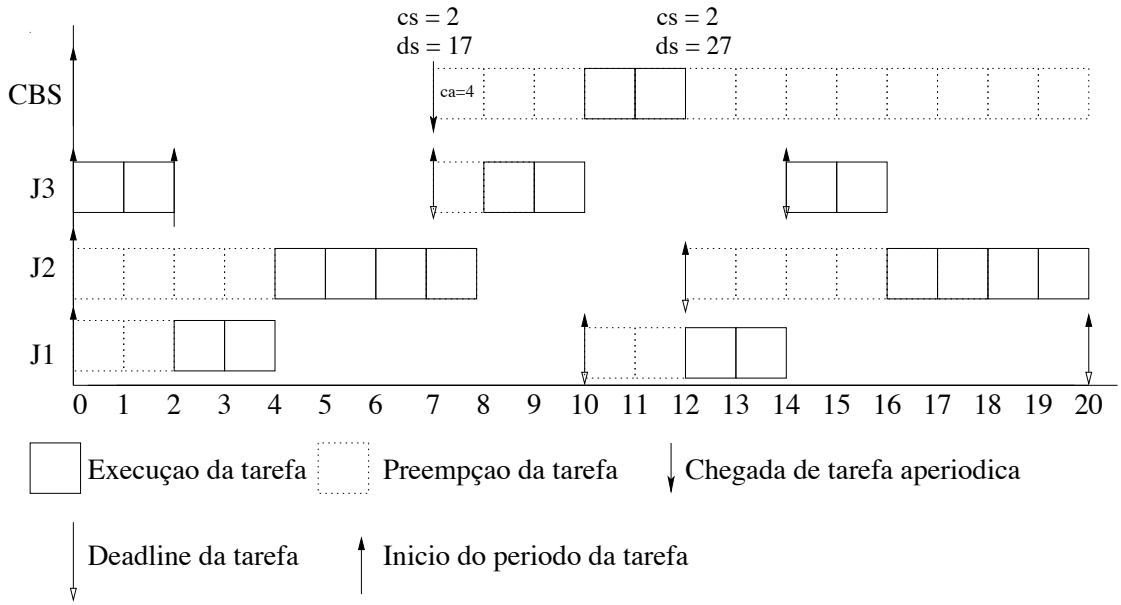


Figura 5: Escalonamento das tarefas j_1, j_2 e j_3 segundo o EDF, com CBS para executar as tarefas aperiódicas

Escalonabilidade do CBS: A utilização do processador do servidor CBS com parâmetros (c_s, p_s) é

$$u_s = \frac{c_s}{p_s}, \tag{2.9}$$

independente dos tempos computacionais e do padrão chegada das tarefas atendidas. Dada uma série de n tarefas periódicas com utilização do processador u_p e um CBS com utilização do processador u_s. O sistema é escalonável se, e somente se, a equação 2.4 for válida (SPURI; BUTTAZZO, 1996).

3 *USO DE SERVIDORES PARA TOLERÂNCIA A FALHAS*

Tolerar falhas significa manter o sistema em funcionamento correto mesmo na presença de erros. Em Sistemas de Tempo Real, contudo, prover tolerância a falhas tem um agravante adicionalse comparadocom sistemas de propósito geral, pois erros são eventos imprevisíveis e existem *deadlines* para serem cumpridos.

Ao se tentar recuperar o sistema de um dado erro, por exemplo, corre-se o risco de causar interferência em tarefas menos prioritárias, fazendo com que as mesmas percam seus *deadlines*. Obviamente, isto é indesejável.

Como visto no capítulo anterior, servidores provêm isolamento entre a execução de tarefas aperiódicas sobre tarefas periódicas com *deadlines* críticos. Comparando-se tarefas aperiódicas e rotinas de recuperação de erros, percebe-se que há alguma semelhança: tais rotinas podem ocorrer a qualquer momento (devido à ocorrência de erros) e sua execução não pode interferir no cumprimento dos *deadlines* das demais tarefas do sistema. Motivados por esta semelhança, investigamos, neste capítulo, o uso de servidores para escalonar rotinas de recuperação.

Inicialmente, conceitos básicos de tolerância a falhas serão explicados e o modelo de escalonamento assumido será descrito. A investigação será baseada em simulação.

3.1 TOLERÂNCIA A FALHAS

Tolerância a falhas são técnicas que garantem o funcionamento correto do sistema na presença de falhas. Uma abordagem usada é conhecida como detecção e recuperação de erros (LAPRIE, 1992). Em linhas gerais, a *detecção de erro* origina um sinal de erro ou mensagem no sistema (AVIZIENIS A. ; LAPRIE, 2000). Após o erro ser detectado, é necessário que rotinas de recuperação sejam ativadas.

Estas rotinas de *recuperação* transformam o estado inconsistente (ocasionado pela ocorrência de um erro) do sistema num estado consistente. Há duas maneiras de prover recuperação (LA-

PRIE, 1993): *rollback*, onde a transformação do estado do sistema consiste no retorno do sistema ao último estado consistente salvo antes do erro acontecer; *rollforward*, onde o sistema é colocado num estado sem erros detectados. Na nossa monografia usaremos a abordagem *rollforward*. De maneira simplificada, assumimos que a re-execução de tarefas falhas é suficiente para recuperar o sistema.

3.2 MODELO DO SISTEMA USADO NA SIMULAÇÃO

Assumimos o modelo de sistema ilustrado na figura 6. Um escalonador EDF escolhe, de duas filas, qual tarefa irá executar. Há uma fila de tarefas periódicas e uma fila de tarefas aperiódicas. Esta é controlada por um servidor de tarefas aperiódicas. As tarefas aperiódicas submetidas ao sistema são tarefas de recuperação (re-execução das tarefas que sofreram erros durante sua execução). Baseado neste modelo, variamos o tipo de servidor usado a fim de avaliar suas características com relação à capacidade do sistema em executar rotinas de recuperação. Cada servidor escalona sua fila de tarefas aperiódicas segundo o algoritmo EDF.

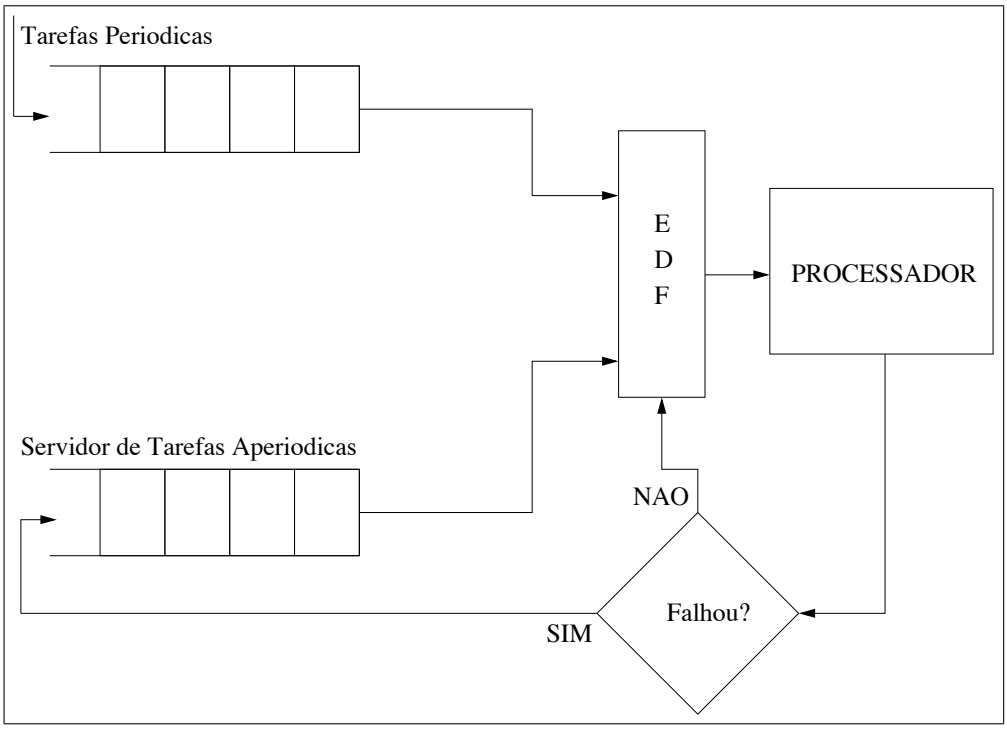


Figura 6: Modelo de escalonamento assumido

3.3 DESCRIÇÃO DA SIMULAÇÃO

O simulador foi desenvolvido em *java*, usando *NetBeans IDE 4.0* rodando a máquina virtual *java* versão *1.5*.

O conjunto de tarefas simuladas continha nove tarefas periódicas e o tempo de simulação considerado foi de *10.000* unidades de tempo. Desta forma, a simulação foi longa o suficiente para observarmos o comportamento dos servidores.

A utilização de processador do conjunto de tarefas simuladas foi 89%. A ocorrência de erros foi simulado da seguinte forma: Durante o período de simulação, 10% das unidades de tempo foram escolhidas aleatoriamente, distribuídas de forma uniforme ao longo do tempo, para representar erros.

A tarefa em execução no instante do erro sofrerá interrupção e uma rotina de recuperação (re-execução) será submetida.

Não foram consideradas na simulação, tempo de troca de contexto ou tempo de detecção de erro. Estes foram considerados instantâneos.

Para efeitos de comparação, simulamos a recuperação de tarefas usando apenas EDF. Ao utilizarmos o EDF composto com os servidores, quando uma falha acontece, na unidade de tempo que uma tarefa estava executando, esta tarefa é colocada em estado de falha, reiniciada e enfileirada na fila de prontos do servidor como uma tarefa aperiódica. Esta tarefa aperiódica deve cumprir a meta da tarefa periódica que falhou, assim, ela tem as mesmas características, como *deadline*, período e tempo máximo de execução da tarefa periódica. A tarefa periódica voltava a fila de tarefas periódicas ativas, saindo do estado de falha, em seu próximo período, quando esta é reiniciada.

Cada servidor executou suas tarefas aperiódicas segundo suas características explicadas no capítulo anterior. Foram observados quatro tipos de parâmetros de comparação:

1. A *taxa de recuperação* é calculada através da divisão da quantidade de tarefas que recuperaram pela quantidade total de tarefas do sistema.

2. A *taxa de interferência* representa a divisão da quantidade de tarefas que perderam o *deadline*, sem serem afetadas por falhas ao longo de sua execução, pela quantidade total de tarefas do sistema.

3. A *taxa de deadlines perdidos* é obtida através da divisão da quantidade total de tarefas que perderam *deadline*, por qualquer motivo, pela quantidade total de tarefas executadas

no sistema.

4. A *utilização real* representa o percentual do processador utilizado pelo sistema. Esse parâmetro serve para mostrar que, se todas as unidades de tempo foram usadas para executar tarefas, a utilização foi total. Caso contrário, esta utilização consumiu um percentual do potencial de utilização do sistema.

3.4 AVALIAÇÃO DOS RESULTADOS

Como pode ser observado na tabela 1, o escalonador EDF apresenta um alto índice de recuperação, porém, como exemplo, provoca interferência nas tarefas que não foram afetadas por erros. Este comportamento já era esperado, pois não há isolamento entre as tarefas de recuperação e as tarefas periódicas. As tarefas periódicas concorrem pelo processador, junto com as tarefas de recuperação que mantiveram o *deadline* inalterado, obtendo a mesma prioridade inicial.

Política de escalonamento	EDF	DDS	CUS	TBS	CBS
Qtde de tarefas executadas	1242	424	1135	1149	1148
Taxa de recuperação	0,312	0,0287	0,220	0,220	0,266
Taxa de interferência	0,02	0	0	0	0
Taxa de *deadlines* perdidos	0,148	0,559	0,172	0,169	0,161
Utilização real	1	0,3407	0,9845	0,9995	1

Tabela 1: Tabela comparativa de desempenho de diversos servidores, feita a partir de simulação

O DDS, apesar de prover isolamento, apresenta um baixo índice de recuperação, por isso não é indicado para ser usado em sistemas que precisam recuperar suas tarefas falhas. Além disso, ele desperdiça uma grande quantidade de banda, na medida em que só utiliza uma quantidade fixa de tempo por período. Quando sobra unidades de tempo devido a ociosidade do processador num período, essas unidades não são aproveitadas pelo DDS, subutilizando assim o potencial de processamento do sistema.

O CUS apresenta um melhor índice de recuperação do que o DDS, porém ainda não faz uso de toda a banda disponível no sistema. Esse servidor tem a característica de tentar utilizar o máximo de banda possível. Por ser conservador, utiliza apenas o tamanho de banda seguramente disponível, evitando qualquer possibilidade de interferência nas demais tarefas.

O TBS, que é um melhoramento do CUS, tem um índice de recuperação maior, pois usa uma parte maior da banda de utilização disponível do sistema. Realiza sua *carga* com mais frequência, porém ainda não faz utilização de toda a banda. Melhor do que o CUS, devido a

uma pequena, mas importante, modificação nas regras de recarga, este servidor, que realiza sua *carga* assim que possível executa, em alguns casos, algumas unidades de tempo a mais que o CUS.

O CBS foi desenvolvido com a intenção de usar todo o tempo disponível do sistema em prol das tarefas aperiódicas (no caso, tarefas de recuperação). Isto fez com que sua utilização fosse máxima e sua taxa de recuperação crescesse, porém ainda encontra-se inferior a taxa de recuperação do EDF, com a vantagem de não ocasionar interferência no sistema. Este servidor foi o que apresentou melhor taxa de recuperação, com menor taxa de deadlines perdidos.

3.5 CONSIDERAÇÕES

Neste capítulo, examinamos, por simulação, o uso de servidores para tolerância a falhas. Foi observado que o CBS apresentou melhor resposta. Contudo, o nível de recuperação não conseguiu atingir o do algoritmo EDF. Isto pode ser explicado, pois o EDF ocasiona interferência nas tarefas que não falharam, logo utiliza o tempo que deveria ser reservado a estas tarefas para garantir o cumprimento de seus *deadlines*.

Porém, sabe-se que a idéia de usar servidores para tolerância a falhas é potencialmente viável, logo itens como *regras de consumo* e *regras de recarga* devem ser revistos para tentar propor um servidor que apresente um desempenho tão bom quanto o EDF, sem ocasionar interferência nas tarefas periódicas.

4 METODOLOGIA E DIFICULDADES ENCONTRADAS

Inicialmente, foi realizada uma revisão bibliográfica sobre Sistemas de Tempo Real (BURNS; WELLINGS, 2001; LIU, 2000; CORMEN; LEISERSON; RIVEST, 1990; COULOURIS; DOLLIMORE; KINDBERG, 2001). Após ter obtido algum conhecimento na área, decidimos investigar o uso de servidores para o tratamento de falhas em Sistemas de Tempo Real. Em seguida, uma nova fase do estudo foi iniciada (BENNETT; ZHANG, 1997; BERNSTEIN, 1997; ABENI; BUTTAZZO, 2004; SPURI; BUTTAZZO, 1996; CLARK; SHENKER; ZHANG, 1992; DEMERS; KESHAV; SHENKER, 1989; DENG; SUN, 1997; DENG; LIU, 1997; LIU; LAYLAND, 1973; AVIZIENIS A. ; LAPRIE, 2000; CANDEA, 2003; LAPRIE, 1992, 1993).

Houve então a necessidade de se aprofundar nas técnicas de escalonamento. Concluídos os estudos, passamos a fase de implementação dos servidores escolhidos.

A fase seguinte foi focada nas discussões dos resultados obtidos na simulação. Finalmente, iniciamos a escrita da monografia. Com a ajuda de anotações que fizemos ao longo do processo, com o intuito de facilitar a posterior escrita, desenvolvemos a monografia com reuniões semanais para discutir o andamento, até a conclusão desta.

Ao longo do desenvolvimento do projeto, reuniões periódicas com o professor orientador foram realizadas. Além disso, um seminário foi apresentado.

As principais dificuldades encontradas foram associadas ao entendimento das políticas de escalonamento estudadas. Tais políticas, geralmente, são embasadas em modelos matemáticos com alto nível de complexidade.

5 CONSIDERAÇÕES FINAIS

A teoria de escalonamento em Sistemas de Tempo Real é bastante complexa e apresenta muitas abordagem de escalonadores para diferentes propósitos. Neste estudo, apresentamos uma verificação do uso de várias técnicas de uma das abordagens: servidores de tarefas aperiódicas.

Servidores são comumente usados para prover isolamento temporal entre as tarefas críticas e não-críticas. Seu uso para tolerância a falhas ainda não havia sido realizado de maneira satisfatória em sistemas baseados em escalonamento dinâmico.

A simulação realizada focou em alguns dos mais conhecidos servidores. Os resultados encontrados indicam, em geral, que não há ganhos significativos no uso de servidores para tolerância a falhas, apesar destes proverem o desejado isolamento temporal entre tarefas críticas e de recuperação.

Acreditamos que o estudo realizado possa servir de base para futuras pesquisas, onde os aspectos específicos de tolerância a falhas possam ser levados em consideração na definição das regras dos servidores.

Neste contexto, pretendemos nos aprofundar no estudo aqui iniciado numa futura fase da pesquisa. Principalmente, iremos ampliar as considerações até então realizadas, abordando outros tipos de servidores. Posteriormente, possíveis adaptação dos servidores estudados poderão ser considerados.

REFERÊNCIAS

ABENI, L.; BUTTAZZO, G. Resource reservation in dynamic real-time systems. *Real-Time Syst.*, Kluwer Academic Publishers, Norwell, MA, USA, v. 27, n. 2, p. 123–167, 2004. ISSN 0922-6443.

AVIZIENIS A. ; LAPRIE, J. . R. B. Fundamental concepts of dependability. In: IEEE. *Proceedings of ISW 2000. 34th Information Survivability Workshop*. 2000. p. 7–12. Disponível em: <http://citeseer.nj.nec.com/avizienis01fundamental.html>.

BENNETT, J. C. R.; ZHANG, H. Hierarchical packet fair queueing algorithms. *IEEE/ACM Trans. Netw.*, ACM Press, v. 5, n. 5, p. 675–689, 1997. ISSN 1063-6692.

BERNSTEIN, P. A. *Principles of Transaction Processing*. [S.l.: s.n.], 1997.

BURNS, A.; WELLINGS, A. *Real-Time Systems ans Programming Languages*. third. [S.l.]: Addison Wesley, 2001.

CANDEA, G. The basics of dependability. *Stanford University*, 2003.

CLARK, D. D.; SHENKER, S.; ZHANG, L. Supporting real-time applications in an integrated services packet network: Architecture and mechanism. In: *Proc. of ACM SIGCOMM*. [s.n.], 1992. p. 14–26. Disponível em: <http://citeseer.ist.psu.edu/clark92supporting.html>.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L. *Introduction to Algorithms*. [S.l.]: /, 1990.

COULOURIS, G.; DOLLIMORE, J.; KINDBERG, T. *Distributed Systems — Concepts and Design*. 3. ed. aw, 2001. Disponível em: <http://www.cdk3.net/>.

DEMERS, A.; KESHAV, S.; SHENKER, S. Analysis and simulation of a fair queueing algorithm. *Proceedings of the SIGOCMM '89 Symposium on Communications Architectures and Protocols*, v. 19, n. 4, p. 1–12, set. 1989. Part of ACM SIGCOMM Computer Communication Review.

DENG, Z.; LIU, J. W.-S. Scheduling real-time applications in an open environment. In: *RTSS '97: Proceedings of the 18th IEEE Real-Time Systems Symposium (RTSS '97)*. Washington, DC, USA: IEEE Computer Society, 1997. p. 308. ISBN 0-8186-8268-X.

DENG, Z.; LIU, J. W.-S.; SUN, J. A scheme for scheduling hard real-time applications in open system environment. In: *Proceedings of Ninth Euromicro Workshop on Real-Time Systems*. Toledo, Spain: IEEE Computer Society, 1997.

LAPRIE, J.-C. (Ed.). *Dependability: Basic Concepts and Terminology*. [S.l.: s.n.], 1992. (Dependable Computing and Fault Tolerance, v. 5).

LAPRIE, J. C. Dependability: from concepts to limits. *12th IFAC International Conference on Computer Safety, Reliability and Security (SAFECOMP'93)*, v. 1, n. 1, p. 157–168, Octobre 1993. Communication invitée.

LIU, C. L.; LAYLAND, J. W. Scheduling Algorithms for Multiprogramming in a Hard Real-Time Environnment. *Journal of the Association for Computing Machinery*, v. 20, n. 1, p. 46–61, jan. 1973.

LIU, J. *Real-Time Systems*. first. [S.l.]: Prentice Hall, 2000.

SPURI, M.; BUTTAZZO, G. Scheduling aperiodic tasks in dynamic priority systems. In: . [S.l.]: Real-Time Systems, 1996. v. 10, n. 2, p. 179–210.